University of Puerto Rico
Mayagüez Campus
Mayagüez, Puerto Rico

Department of Electrical and Computer Engineering

Programming Assignment 2- Agents

by

Diego H. Merchan Rueda
Marco A. Monserrat Montoto
Edgard A. Díaz Bobé
Malik J. Paramo Rios

For: Professor Jose F. Vega Riveros
Course: ICOM5015, Section 001D
Date: March 6, 2026

Introduction

Purpose of Assignment

An A.I. Agent, whether implemented as software or abstract models, continuously receives information from its environment and selects the most appropriate actions that aim to improve its performance according to their predetermined measurements. The interaction between perception, action, and evaluation is what forms the foundation of artificial intelligence, as its goal is to design an agent that behaves rationally under the selected conditions.

This is what PEAS framework is all about: Performance measurement, Environment, Actuators, and Sensors. The PEAS offers a well-built manner in which an agent must accomplish its goal, and the constraint under which it will operate. Different agent architectures, such as random agents, reflex agents, and agents with internal state embody the different assumptions in which information is shown, and how decisions are made under these conditions. As defined by Russell and Norvig, an intelligent agent is a system that perceives its environment and selects actions that are expected to maximize its performance over time [1].Comparing these architectures will highlight their strength and differentiate them, especially when their environments become more complex.

Hypothesis

We hypothesize that the SimpleReflexAgent will outperform the RandomAgent in the two-location world due to it always acting on visible dirt immediately and never wasting actions. In the 2D world, the model-based exploration agent is expected to outperform the other two agents, since systematic exploration eliminates the need for redundant moves. It is expected that the margin of difference between the model-based exploration agent and ReflexAgent will not be wide due to the map consisting of very small grids where random coverage is similarly efficient in comparison to systematic coverage.

Experimental Design Choice

The experiment uses a randomized fixed-horizon comparative design which isolates the effect of agent policies on performance in the two-location vacuum world. For this experiment, both the Simple-Reflex Agent and the Random Agent operate under identical environmental conditions: the same locations(A and B), the same transition model, the same percept structure and the same action set. By making these elements constant, the experiment ensures that any differences in performance arise solely from the agent's decision-making and rules, rather than from variables.

Each trial begins with randomized initial conditions, including the agents starting locations as well as the dirt configuration across both squares. A fixed time horizon is applied at the start of each run so that cumulative performance scores are directly comparable between the agents. This process is repeated across a large number of trials to obtain the most stable estimates of central tendency.

The primary evaluation metric is the final cumulative performance score, which reflects both cleaning success and cost of movement. The criterion is to prefer the agent that shows a consistently higher mean and median performance with lower amount of variability. This design minimizes confounding factors, isolates policy quality and provides statistical evidence for both agents, making it perfect for comparing the agents rationality.

Methodology

This experiment was structured to support a controlled and reproducible comparison between a ***Simple-Reflex Agent*** and a ***Random Agent*** within a classic two-location vacuum world. This was implemented using a modular architecture that separates the environment model, perception, action execution, agent program and performance evaluation. This was done in order to ensure that both agents interacted with the same environment, allowing policy differences to be isolated as the sole experimental factor.

Environment and Simulation Framework

The environment consists of two connected locations (A and B), in which each may begin in a clean or dirty state. At the start of the trial, both the agent's initial position and the dirt configuration are randomized to avoid bias toward any particular starting condition. The transition model is deterministic, in which movement actions relocate the agent to the adjacent square, ***SUCK*** removes dirt if present, and ***NOOP*** leaves the state unchanged. A fixed time horizon is applied to all runs, ensuring that cumulative performance scores are directly comparable across agents. The simulation loop follows the standard interaction cycle in which the environment generates a percept based on the current state, the agent selects an action according to its policy, the environment applies the transition function, and the performance measure is updated. This cycle repeats until the horizon is reached.

Agent Implementations

The ***Simple-Reflex Agent*** follows a deterministic rule set: if the current square is dirty, it selects SUCK; otherwise, it moves to the opposite location. While the ***Random Agent*** samples uniformly from the available actions at each step, independent of percepts. Both agents operate under the same percept structure location and dirt status, ensuring that differences in performance arise from policy design rather than information availability.

Performance Evaluation

Performance is measured cumulatively over the fixed horizon. The scoring function rewards cleaning actions and penalizes movement, reflecting the goal of efficient cleaning with minimal unnecessary motion. For each agent, the experiment is repeated across many randomized trials to obtain stable estimates of mean performance, median performance, and variability. These results provide both central-tendency evidence and stableness indicators, allowing the experiment to determine which policy performs more consistently under randomized initial conditions.

Results and Analysis

Table 1 shows the step-by-step scores for both agents on the same initial state, where both agents A and B are dirty, start at A, and have 20 steps.

Agent	Steps Run	Final Score	FinalState
Simple Reflex Agent	20	28	Both clean
Random Agent	20	12	Both clean

Fig. 01, Table 1, Single Run Results

The Simple Reflex Agent cleaned location A right away at step 0, which gave it the +5 bonus, then moved to B and cleaned it at step 2. From step 3 on, both squares stayed clean, and the agent kept gaining a net +1 per step, since it got +2 for two clean squares and lost −1 for the action cost, finishing with 28. The RandomAgent did not clean A until step 2 and then spent steps 3 through 18 moving around without cleaning dirty B, making unnecessary bounces, LEFT moves into the wall, and two NOOP actions before finally cleaning B at step 19. That gave it a final score of 12, which is 57% lower than the reflex agent.

Agent	Trials	Average Score
Simple Reflex Agent	50	34.18
Random Agent	50	32.84

Fig. 02, Table 2, Multiple Trial Average

Across 50 trials with randomized initial states, the gap became much smaller compared to the single run. This makes sense because when one or both squares already start clean, the advantage of cleaning right away is not as big, and both agents can collect similar rewards from squares that are already clean. The reflex agent still comes out on top, which supports the hypothesis, but the small difference also shows that the two-location world is simple enough that even a random policy can usually finish the task within 30 steps.

Agent	Mean	Std Dev	Min	Max
Berkeley Reflex	2462.25	712.20	1044	4280
Berkeley Model-Based	2322.38	817.71	894	4425
Berkeley Random	1970.75	624.79	884	3407

Fig. 03, Table 3, 2D Grid Multi-Trial Results

The Berkeley Random agent finished last with a mean of 1970.75, leaving the biggest gap compared to the other two. In a 4 to 7 by 4 to 7 grid with obstacles, choosing randomly from all six actions means many moves end up being wasted, especially when UP or DOWN may be blocked. Since the agent has no way to remember failed moves or avoid repeating them, that waste keeps adding up across 150 steps and uses part of the action budget on moves that do not help. The Berkeley Reflex agent came out first with a mean of 2462.25. Since it always gives priority to SUCK whenever the current square is dirty, it is able to collect the +5 cleaning bonus quickly and then continue moving through the grid. In environments of this size and with this step limit, that simple greedy rule was enough to beat the model-based agent on average. The Berkeley Model-Based agent placed second with 2322.38, but it also had the highest standard deviation at 817.71 and the highest single-trial maximum at 4425. The extra work of DFS backtracking and keeping track of unexplored cells can cost more steps in smaller or more open grids, where random movement may still reach dirty cells quickly. Even so, its higher maximum score suggests that it can do better in larger or more restricted environments, which matches what was seen later in the corridor experiment.

Agent	Score
Reflex with state explore	3279
Randomized Reflex	1939
Randomized	1458

Fig. 04, Table 4, Corridor Bad Environment 1×18 grid, 200 steps, 70% dirt probability

In the 1 by 18 corridor, the model-based agent clearly performed the best with a score of 3279. It moved through the corridor from left to right on its first pass, cleaned each dirty cell it found, and once it learned that UP and DOWN were blocked, it stopped wasting steps in those directions. The deterministic reflex agent scored 1939 because it at least kept moving between LEFT and RIGHT in a consistent way, but its random movement still caused a lot of back-and-forth over cells that were already clean. The randomized reflex agent got the lowest score at 1458 because it kept choosing UP and DOWN, which wasted about one-third of its 200 steps on blocked moves, and its random bouncing made it take much longer to cover the full corridor than the model-based agent's more organized sweep.

Conclusions

The assignment implemented a complete PEAS vacuum world simulator spanning through two exercises. They both shared the same performance measure, sensor interface, simulator infrastructure, while having different environments and agents. The results confirmed the main hypothesis for the two-location world, where the Reflex agent outperformed the random agent by reacting immediately to visible dirt. As for the 2-D environment, the results varied, the model-based exploration agent performed better in the adversarial corridor since learning to avoid blocked directions significantly reduced wasted steps. But on small random grids the advantage was gained by the Reflex agent since the methodological exploration worked against it when it came to movement efficiency. In other words the reflex agent achieved more by sheer probability while the Model-based agent lost points for being overly detailed.

Lessons Learned

PEAS separation helped enforce better software design by keeping every section as independent classes, facilitating adding new code. How scoring is done affected how agents were ranked, the continuous per step reward gave advantage for cleaning early and masked differences between agents on the longer runs. Model-based agents performed better in constrained geometries, averaging across many small grids deluded the advantage this kind of agent possessed over the others. Random baselines in small worlds benefit random movers over the others, since they will cover most cells.Though a bigger world and complexity give more value to deliberate exploration.

References

[1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
[2] Python Software Foundation, *Python 3 Documentation*.[Online].Available: https://docs.python.org/3/

Group Collaboration

1. Diego - Worked on the exercise 2.11 and wrote Methodology.
2. Malik -Worked on the exercise 2.11 and wrote Experimental Design Choice.
3. Edgard - Worked on the exercise 2.14 and wrote Results and analysis.
4. Marco - Worked on the exercise 2.14 and wrote Introduction and Conclusion.